

Day 3 SDE-2 Interview Guide: Spring AOP, Proxies & Self-Invocation

Target Roles: SDE-2 / Senior Backend | Project: Digital Wallet & Payment Ledger
Track: Core Spring Internals (AOP & Proxy Mechanics) & Project Architecture Fixes

Part 1: Spring AOP & Proxy Mechanics

Q1. What is Spring AOP, and how is it different from AspectJ?

Simple Understanding

AOP = Aspect-Oriented Programming.

The idea is:

Instead of writing the same supporting logic inside every business method, we define it separately and let the framework apply it automatically.

Examples of cross-cutting concerns:

- Transactions
- Logging
- Security
- Caching
- Retry
- Asynchronous execution

For example:

```
@Transactional
public void transferMoney() {
    // business logic
}
```

We don't manually write:

```
beginTransaction();
```

```
try {
    // business logic
    commit();
} catch (Exception e) {
    rollback();
}
```

Spring AOP can apply this behavior around the method.

How Spring AOP works

Spring AOP is primarily **proxy-based**.

Conceptually:

Controller



Spring Proxy



AOP Interceptor



Real Service



Business Method

For @Transactional, the interceptor can:

Start transaction



Call real method



Success? → Commit

Failure? → Rollback

Important limitation

Spring AOP primarily intercepts **method executions on Spring-managed beans through their proxies**.

Spring AOP vs AspectJ

AspectJ is more powerful.

Instead of depending only on a proxy, AspectJ can modify the actual bytecode through weaving.

AspectJ supports:

- Method execution
- Constructor execution/calls
- Field access
- Objects that are not Spring beans
- Compile-Time Weaving (CTW)
- Load-Time Weaving (LTW)

Easy difference

Spring AOP



Proxy around Spring bean



Intercept method call

versus:

AspectJ

↓

Modify/weave bytecode

↓

Can intercept much more than proxy-based calls

Interview Answer

Spring AOP is primarily a proxy-based AOP mechanism used to apply cross-cutting concerns such as transactions, security, caching, and asynchronous execution to Spring-managed beans. AspectJ is a more powerful weaving-based AOP implementation that can modify bytecode at compile time or load time and supports join points beyond normal Spring proxy method interception.

Q2. What is the difference between JDK Dynamic Proxy and CGLIB Proxy?

Spring can create proxies in different ways.

1. JDK Dynamic Proxy

JDK Dynamic Proxy is a Java feature provided by:

`java.lang.reflect.Proxy`

The target class must implement an interface.

Example:

```
public interface PaymentService {  
    void pay();  
}  
  
@Service  
public class PaymentServiceImpl implements PaymentService {
```

```
    @Transactional  
    public void pay() {  
        // payment  
    }  
}
```

Conceptually:

PaymentService interface

↑

|

JDK Dynamic Proxy



PaymentServiceImpl

The method invocation is handled through an InvocationHandler.

2. CGLIB / Class-Based Proxy

A class-based proxy creates a subclass of the target class.

Example:

@Service

```
public class PaymentService {
```

```
    @Transactional
```

```
    public void pay() {
```

```
        // payment
```

```
    }
```

```
}
```

No interface is required.

Conceptually:

PaymentService



Generated Proxy subclass

The proxy can intercept methods that it can override.

Therefore:

```
final class PaymentService
```

cannot be proxied through subclassing.

Similarly, a:

```
final void pay()
```

method cannot be overridden by the generated subclass.

Main Difference

Feature	JDK Dynamic Proxy	Class-based Proxy
Requires interface?	Yes	No
Proxies	Interfaces	Classes

Feature	JDK Dynamic Proxy	Class-based Proxy
Mechanism	Proxy + InvocationHandler	Generated subclass
Final class	Not applicable to target-class subclassing	Cannot subclass
Final method	Interface method dispatch has its own rules; class-based final methods cannot be overridden	Cannot be overridden/intercepted through subclassing

Spring Boot

In Spring Boot applications, class-based proxying is commonly enabled by default through:

`spring.aop.proxy-target-class=true`

So you will commonly encounter class-based proxies even when your service does not implement an interface.

Interview Answer

JDK Dynamic Proxy works through Java interfaces and requires the target to implement an interface.

Class-based proxying creates a subclass of the target class, so an interface is not required, but final classes and final methods cannot be overridden and therefore cannot be intercepted through that mechanism.

Q3. What is the Self-Invocation Problem in Spring AOP?

This is one of the most important concepts for our project.

First understand the normal case

Suppose:

`@Service`

```
public class WalletService {
```

```
    @Transactional
```

```
    public void deductMoney() {
```

```
        // database operation
```

```
    }
```

```
}
```

The Controller does:

```
walletService.deductMoney();
```

The object injected into the Controller is conceptually:

Controller

↓

WalletService Proxy

↓

@Transactional interceptor

↓

Real WalletService

↓

deductMoney()

So Spring gets a chance to apply the transaction.

Now the Problem

Suppose the same class has:

@Service

public class WalletService {

```
    public void transfer() {  
        this.deductMoney();  
    }
```

@Transactional

```
    public void deductMoney() {  
        // database operation  
    }
```

}

Controller calls:

walletService.transfer();

The first call goes through the proxy:

Controller

↓

Proxy

↓

transfer()

But inside transfer():

this.deductMoney();

The call goes directly to the target object.

Real WalletService

↓

this.deductMoney()

↓

deductMoney()

The proxy is not involved.

Therefore the `@Transactional` interceptor doesn't get a chance to apply the transaction boundary for that invocation.

Important Point

These two are effectively the same regarding self-invocation:

`this.deductMoney();`

and:

`deductMoney();`

when called from another method in the same class.

Both are internal calls on the current object and bypass the Spring proxy.

Why is this called "self-invocation"?

Because the object is essentially saying:

"I am calling another method on myself."

WalletService

↓

calls itself

↓

WalletService

Hence:

Self-invocation.

What annotations can be affected?

Depending on the annotation and configuration, proxy-based advice can be bypassed for:

- `@Transactional`
- `@Async`
- `@Cacheable`
- `@Retryable`
- Custom Spring AOP aspects

Interview Answer

Self-invocation occurs when one method of a Spring bean calls another method of the same bean directly, such as `this.method()`. Since the internal call goes directly to the target object instead of going through the Spring proxy, proxy-based AOP advice such as `@Transactional` or `@Async` is not applied to that invocation.

Q4. How do you solve the Self-Invocation Problem?

There are several approaches.

Approach 1 — Extract a Collaborator Service ★ Recommended

This is the cleanest architectural solution.

Instead of:

WalletTransferService

```
└─ processTransfer()
└─ executeTransfer()
```

we create:

WalletTransferService

↓

TransactionExecutionEngine

Example:

@Service

```
public class TransactionExecutionEngine {
```

```
    @Transactional
```

```
    public void executeTransferStep() {
```

```
        // transactional work
```

```
    }
```

```
}
```

Then:

@Service

```
public class WalletTransferService {
```

```
    private final TransactionExecutionEngine executionEngine;
```

```
    public WalletTransferService(TransactionExecutionEngine executionEngine) {
```

```
        this.executionEngine = executionEngine;
```

```
    }
```

```
    public void processTransfer() {
```

```
        executionEngine.executeTransferStep();
```

```
    }
```

```
}
```

Now:

WalletTransferService

↓

TransactionExecutionEngine Proxy

↓

@Transactional

↓

executeTransferStep()

The call crosses a Spring bean boundary and reaches the collaborator through its proxy.

Why this is preferred

It gives us:

- Cleaner responsibility separation
- Better testability
- Less coupling
- Easier maintenance
- No self-reference
- No proxy-specific code in business logic

Approach 2 — Self-Injection

You can inject the Spring-managed proxy of the same bean and call through that reference.

For example, conceptually:

@Service

public class WalletService {

private final ObjectProvider<WalletService> self;

public WalletService(ObjectProvider<WalletService> self) {
 this.self = self;
}

public void transfer() {
 self.getObject().deductMoney();
}

@Transactional
public void deductMoney() {
 // transactional work
}

```
}
```

Now:

WalletService

↓

ObjectProvider

↓

Spring proxy

↓

deductMoney()

So the proxy can apply the transaction advice.

But...

This is generally less clean than extracting a collaborator.

It introduces a self-reference and makes the code more dependent on Spring proxy behavior.

Approach 3 — `AopContext.currentProxy()`

Spring can expose the current proxy.

Then code can conceptually do:

```
((WalletService) AopContext.currentProxy())  
    .deductMoney();
```

with proxy exposure enabled.

However, this couples business code directly to Spring AOP infrastructure.

Therefore:

Useful in some cases, but generally not the first architectural choice.

Approach 4 — AspectJ Weaving

Instead of relying on Spring's proxy mechanism, AspectJ can weave the advice into the bytecode.

Therefore, proxy self-invocation limitations can be avoided.

But this is a much heavier solution and is usually unnecessary for normal Spring Boot applications.

Q5. Does `@Transactional` work on private or protected methods?

The safe interview answer

With normal **Spring proxy-based `@Transactional`**, do not rely on `@Transactional` for private methods.

Private methods cannot be overridden by a class-based proxy, so proxy-based interception cannot be applied to them.

For normal Spring AOP usage, put transactional boundaries on **public service methods**.

What about protected/package-private?

This needs a little more nuance.

The exact visibility rules depend on the proxy mechanism and Spring configuration/version.

So avoid saying:

"JDK and CGLIB can only intercept public methods."

That statement is too broad.

For SDE-2 interviews, the practical recommendation is:

@Transactional

```
public void transfer() {  
    // business operation  
}
```

Keep the transaction boundary on a public service method.

Interview Answer

With standard Spring proxy-based transaction management, private methods cannot be intercepted as transactional methods. For reliable and conventional Spring transaction management, transaction boundaries should be placed on public service methods. If interception of methods beyond normal proxy limitations is required, AspectJ weaving can be considered.

Part 2: Wallet Project Architecture & Transaction Drill

Q6. What happened in WalletSelfInvocationTest before the fix?

We had:

```
public void processTransferWithSelfInvocation(...) {  
  
    this.executeTransferStep(...);  
}
```

@Transactional

```
public void executeTransferStep(...) {  
  
    sender.debit(amount);  
  
    walletRepository.save(sender);  
  
    throw new RuntimeException();  
}
```

Initial state:

Sender = \$100

Receiver = \$0

Transfer:

\$40

The application did:

Sender \$100

↓

Debit \$40

↓

Sender \$60

↓

Exception

Normally, if the @Transactional advice had established the transaction:

\$100

↓

Debit \$40

↓

Exception

↓

ROLLBACK

↓

\$100

But because of:

`this.executeTransferStep(...)`

the proxy was bypassed.

In our test setup, there was no surrounding transaction to roll back the debit.

Therefore:

Sender = \$60

Receiver = \$0

The result is a **partial transfer** and therefore a serious financial consistency problem.

Important correction

Don't say:

"Self-invocation always commits the database."

Instead say:

"Self-invocation prevents the @Transactional advice on the called method from being applied. The actual transaction behavior depends on whether another transaction already exists. In our test there was no outer transaction, so the debit remained committed."

That is the technically correct SDE-2 answer.

Q7. How did TransactionExecutionEngine fix the problem?

We moved:

`executeTransferStep()`

into:

`TransactionExecutionEngine`

Now:

`WalletTransferService`

calls:

`executionEngine.executeTransferStep();`

The call is no longer:

`this.executeTransferStep();`

Instead it is:

`WalletTransferService`

↓

`TransactionExecutionEngine Proxy`

↓

`@Transactional` interceptor

↓

`TransactionExecutionEngine`

↓

`executeTransferStep()`

Spring can now establish the transaction.

Flow:

Initial

Sender = \$100

Receiver = \$0

↓

Start transaction

↓

Debit \$40

Sender = \$60

↓

RuntimeException

↓

TransactionInterceptor sees exception

↓

ROLLBACK

↓

Database restored

Sender = \$100

Receiver = \$0

Interview Answer

We extracted the transactional operation into a separate Spring-managed `TransactionExecutionEngine`. `WalletTransferService` invokes it through the injected bean reference, so the invocation goes through Spring's proxy. The `@Transactional` interceptor starts the transaction and rolls it back when the simulated runtime exception occurs, leaving the sender balance unchanged.

Q8. What is the default rollback behavior of @Transactional?

By default, Spring rolls back for:

RuntimeException

Error

It does **not** automatically roll back for ordinary checked exceptions.

Example:

`@Transactional`

`public void transfer() throws Exception {`

`// database operation`

```
    throw new Exception("Checked exception");
}
```

By default, a checked Exception does not trigger rollback.

You can explicitly configure:

```
@Transactional(rollbackFor = Exception.class)
```

Then checked exceptions can also trigger rollback.

Important interview nuance

Don't say:

"SDE-2 best practice is always to use rollbackFor = Exception.class."

That is too absolute.

Instead:

"Use rollbackFor when the application's business requirements require rollback for checked exceptions."

For example:

```
@Transactional(
    rollbackFor = PaymentProcessingException.class
)
```

can be better than blindly rolling back for every Exception.

Interview Answer

By default, Spring rolls back transactions for unchecked exceptions such as RuntimeException and for Error. Checked exceptions do not trigger rollback by default. If a checked business exception must also cause rollback, we can configure rollbackFor.

Q9. What happens if a @Transactional method is final with class-based proxying?

Suppose:

```
@Transactional
public final void transfer() {
    // business logic
}
```

Class-based proxying works by creating a subclass.

Conceptually:

Original class

↑

|

Generated proxy subclass

The proxy needs to override the method to insert interception logic.

But Java doesn't allow:

```
@Override
```

```
public final void transfer()
```

because a final method cannot be overridden.

Therefore, the class-based proxy cannot apply the normal method interception to that final method.

Interview Answer

With class-based proxying, a final method cannot be overridden by the generated subclass. Therefore Spring cannot apply normal proxy-based interception to that method, so `@Transactional` should not be placed on final methods when relying on class-based proxies.

Q10. How does `@Async` behave with self-invocation?

The concept is exactly the same.

Example:

```
@Service
```

```
public class EmailService {
```

```
    public void process() {
```

```
        this.sendEmail();
```

```
    }
```

```
    @Async
```

```
    public void sendEmail() {
```

```
        // send email
```

```
    }
```

```
}
```

You might expect:

Main Thread

↓

Spring Proxy

↓

`@Async` interceptor

↓

Thread Pool

↓

`sendEmail()`

But because:

`this.sendEmail();`

is a self-invocation:

Main Thread



`process()`



`this.sendEmail()`



`sendEmail()`

The proxy is bypassed.

Therefore, the `@Async` interceptor does not submit the method to the executor.

The method executes **synchronously on the calling thread**.

Interview Answer

`@Async` has the same proxy-based self-invocation limitation. When an `@Async` method is called through the Spring proxy, Spring can submit it to the configured executor. When it is called internally using `this.method()`, the proxy is bypassed, so the asynchronous interceptor is not triggered and the method executes synchronously.

★ Final Mental Model

This is the most important thing to remember from Day 3.

Normal external call

Controller



Spring Proxy



AOP Interceptor



Real Service



Method

Therefore:

`@Transactional` → Transaction can be applied

`@Async` → Async execution can be applied

`@Cacheable` → Caching can be applied

Self-invocation

Real Service

↓

this.method()

↓

Another method in same object

The proxy is skipped.

Therefore proxy-based advice on that internal invocation is skipped.

🔥 One Sentence to Memorize

Spring AOP works through proxies. When a method is called through the Spring proxy, AOP advice can be applied; when a method calls another method on the same object directly, the proxy is bypassed, causing proxy-based annotations such as @Transactional and @Async to be skipped for that invocation.

Interview Flow You Should Say

If the interviewer asks:

"Why did your transaction not roll back?"

Answer:

"The issue was self-invocation. WalletTransferService was calling its own @Transactional method using this.executeTransferStep(). Since Spring AOP is proxy-based, that internal call bypassed the Spring proxy, so the TransactionInterceptor was never applied to that invocation. We fixed it by extracting the transactional operation into a separate TransactionExecutionEngine service, ensuring the call crosses a Spring bean boundary and goes through the transactional proxy."

That's a **strong SDE-2 answer**.